Step 2 - Propose High-Level Design and Get Buy-In

At a high level, the autocomplete system is divided into two major components:

- **Data Gathering Service**
- **Query Service**

High-Level Design

1. Data Gathering Service

The data gathering service collects user search queries and aggregates them in real time.

Whenever a user searches for something, the query string and its frequency count are updated in the system.

Although real-time processing is acceptable for a simple design, it becomes difficult to scale for very large datasets. More scalable approaches will be discussed later in the deep dive section.

Data Gathering Service Workflow

Assume we maintain a **frequency table** that stores:

Query	Frequency

Initially, the frequency table is empty.

Users enter the following queries sequentially:

1. `"twitch"`

2. `"twitter"`
3. `"twitter"`
4. `"twillo"`

After processing these queries, the frequency table becomes:

Query	Frequency
twitch	1
twitter	2
twillo	1

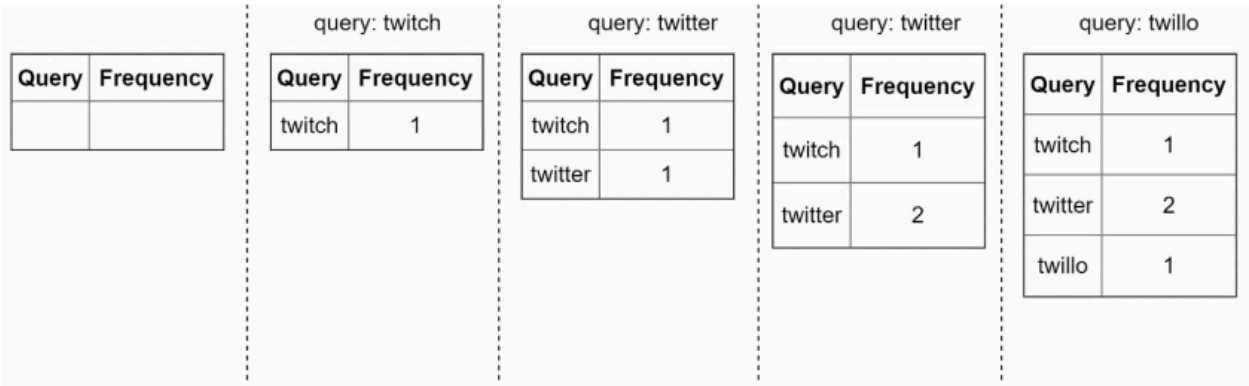

Figure illustrates how the frequency table is updated as users continuously submit search queries.

2. Query Service

The query service returns the **top 5 most frequently searched queries** for a given search prefix.

Assume we have the following frequency table:

Query	Frequency
twitter	35
twitch	29
twillo	25
twin peak	21
twitch prime	18

When a user types:

```
None
tw
```

The system returns the top 5 matching queries sorted by frequency as shown in **Figure**.

SQL Query for Top 5 Suggestions

To retrieve the top 5 most searched queries matching a prefix, we can execute:

```sql
SELECT *

FROM frequency_table

WHERE query LIKE 'tw%'

ORDER BY frequency DESC

LIMIT 5;
```

Query Service Workflow

1. User types a search prefix.
2. Query service searches the frequency table.
3. Matching queries are sorted by popularity.
4. Top 5 results are returned to the user.

Limitations of This Design

This solution works well when:

- Dataset is small
- Traffic volume is low

However, for large-scale systems:

- Database queries become expensive
- High QPS causes bottlenecks
- Prefix matching becomes slow
- Scalability becomes difficult

Therefore, additional optimizations are needed, which will be discussed in the deep dive section.

Summary

Component	Responsibility
Data Gathering Service	Collect and aggregate search queries
Frequency Table	Store query strings and frequencies
Query Service	Return top 5 autocomplete suggestions

This high-level design provides a simple starting point before moving to scalable production-level optimizations.